Concurrence et synchronisation

Exercice 1 (Recherche de concurence)
On propose pour gérer les accès à des cases mémoires partagées les méthodes suivantes :

```
void entree_section(id)
{
    interesse[id] = true;
    while( tour != id)
    {}
}

void sortie_section(id)
{
    interesse[id] = false;
}

void suivant()
{
    int i = 0;
    while (true)
    {
        while(interesse[i])
        {}
        i = (i+1)%p;
        tour = i;
    }
}
```

Les variables `tour` et `interesse` sont partagées par les $p+1$ fils d'exécution. Parmi ceux-ci, p exécutent la fonction `entree_section` avant d'accéder à la section critique et `sortie_section` après. Le dernier fil d'exécution exécute la fonction `suivant` qui supervise le tour de chacun.

Question 1 : Quelles garanties pouvez vous démontrer sur la section critique ?

Exercice 2 (Variables partagées)
Deux fils d'exécution exécutent le programme suivant :

```
while(i < 10)
{
    i = i+1;
    counter = counter +1;
}
```

Les variables `i` et `counter` sont toutes deux des entiers initialisés à 0. On considère que seules la lecture, l'écriture et l'addition sont des opérations atomiques.
Question 1 : On suppose que seul `counter` est une variable dont la valeur est partagée. Prouver qu'il existe une exécution des deux fils pour laquelle `counter` a pour valeur 10 lorsqu'ils terminent.
Question 2 : Prouver que la valeur de `counter` est toujours inférieure ou égale à 20.
Question 3 : On suppose maintenant que `i` est aussi partagé par les deux fils d'exécution. Prouver que la valeur de `counter` à la fin de l'exécution des deux fils est toujours supérieure ou égale à 1 et que cette borne est atteinte.
Question 4 : Calculer la valeur maximale que peut prendre `counter` à la fin de l'exécution.